

Ejercicio técnico — OrderFlow: Backend Serverless en AWS

Nivel: Senior · **Modalidad:** Take-home · **Stack obligatorio:** TypeScript + AWS CDK

Esta es una prueba **de diseño abierto**: te damos el problema de negocio y el conjunto de tecnologías que debes usar, pero **tú decides la arquitectura, qué recursos usas y cómo los usas**. No hay una única solución correcta; evaluamos tu criterio de ingeniería y cómo lo justificas.

1. Contexto y objetivo

Queremos evaluar tu **dominio real del desarrollo de aplicaciones sobre AWS**: tu capacidad de **diseñar, justificar y desplegar** un backend completo, no solo de escribir una función aislada.

Vas a construir **OrderFlow**, el backend de una tienda. Te entregamos **qué** debe hacer la aplicación y **con qué tecnologías** debes construirla; el **cómo** —arquitectura, modelo de datos, contratos de API, topología de red, seguridad, organización del código y de la infraestructura— es **tu decisión de diseño** y parte central de lo que evaluamos.

2. Descripción de la aplicación

OrderFlow es el backend de una tienda que gestiona el ciclo de vida de **pedidos**. A grandes rasgos, debe cubrir las siguientes necesidades funcionales:

- **Usuarios y autenticación:** los usuarios inician sesión y, una vez autenticados, acceden a las funciones que les correspondan.
- **Gestión de pedidos:** un cliente autenticado puede **crear** pedidos y **consultar** el estado de los suyos.
- **Procesamiento de pedidos:** cada pedido, tras crearse, debe **procesarse** (por ejemplo: validación, cálculo de totales, cambios de estado). Este procesamiento puede no ser instantáneo.
- **Administración:** debe existir una capacidad de **consulta administrativa** de los pedidos, pensada para uso interno.
- **Tareas recurrentes:** la plataforma necesita ejecutar **trabajo periódico** (por ejemplo, un reporte o agregación de métricas).
- **Avisos de cambios de estado:** cuando un pedido cambia de estado debe poder **desencadenarse una reacción** (por ejemplo, una notificación).

Cómo modelas estas necesidades, qué estados maneja un pedido, qué endpoints expones y cómo conectas las piezas es decisión tuya.

3. Stack obligatorio

Tu solución **debe estar construida** con las siguientes tecnologías y **hacer uso real** de cada uno de estos recursos de AWS. **No te indicamos para qué usar cada uno**: es parte del ejercicio que decidas su rol dentro de tu arquitectura y lo justifiques.

Lenguaje e IaC

- **TypeScript** para todo el código de aplicación.
- **AWS CDK (TypeScript)** para definir **toda** la infraestructura como código (despliegue reproducible, sin pasos manuales en la consola de AWS).

Servicios y capacidades de AWS que debes incorporar

- **AWS Lambda**
- **Amazon API Gateway**
- **Amazon SQS**
- **Amazon EventBridge**
- **Amazon RDS**
- **Amazon EC2**
- **Autenticación con login + JWT**
- **Endpoints privados** (acceso privado a uno o más componentes de tu solución)

Puedes incorporar servicios adicionales si tu diseño lo requiere (p. ej. Secrets Manager, VPC, balanceadores, etc.); si lo haces, **justifícalo**. Lo que no puede faltar es el uso efectivo de todos los elementos de la lista anterior.

4. Lo que debes diseñar y decidir

Estas son decisiones **tuyas**. No esperamos una respuesta concreta predefinida, sino criterio y coherencia:

- **Arquitectura general**: cómo se conectan los componentes y por qué.
 - **Modelo de datos** en RDS y cómo se accede a él.
 - **Contratos de API**: qué endpoints expones, públicos y/o privados.
 - **Autenticación y autorización**: cómo emites y validas el JWT y cómo proteges los recursos.
 - **Procesamiento y desacople**: cómo manejas el trabajo asíncrono, los reintentos y los errores.
 - **Topología de red y seguridad**: qué es público, qué es privado, y cómo lo aseguras.
 - **Gestión de secretos y de permisos** (IAM) bajo el principio de mínimo privilegio.
 - **Organización del código y de la infraestructura** (CDK).
-

5. Entregables

Entrega un **repositorio Git** (link o `.zip`) que contenga:

1. **Infraestructura en CDK (TypeScript)** desplegable de forma reproducible.
2. **Código de aplicación** (TypeScript).

3. Lo necesario para **inicializar la base de datos** (esquema y datos mínimos de prueba).
 4. **README** con:
 - Pasos para desplegar y poner en marcha la solución.
 - Cómo **probar el flujo end-to-end** (ejemplos de invocación, p. ej. `curl`/Postman).
 - Cómo verificar las partes **privadas** de tu solución.
 5. **Documento de decisiones de arquitectura** (ADR ligero): qué decisiones tomaste, por qué, qué alternativas descartaste y qué trade-offs asumiste. **Este documento es clave**: aquí demuestras el dominio que queremos evaluar.
-

6. Restricciones y supuestos

- **Sin frontend**: solo backend e infraestructura.
 - Usa clases/instancias económicas (free-tier donde sea posible). No buscamos costo, buscamos diseño.
 - Cualquier supuesto que tomes, **déjalo documentado** en el README/ADR.
 - Si algo no alcanzas a implementar, documéntalo: explica **cómo lo resolverías**.
-

7. Criterios de evaluación

Área	Qué miramos
Diseño y arquitectura	Coherencia de la solución, desacople, límites claros, decisiones bien justificadas.
Uso del stack	Que cada tecnología obligatoria se use de forma pertinente , no forzada.
Networking y seguridad	Manejo de lo público vs. privado, secretos, IAM de mínimo privilegio.
Calidad de código (TS)	Tipado, estructura, manejo de errores, claridad.
IaC (CDK)	Reproducibilidad, organización, sin pasos manuales.
Robustez	Manejo de errores, reintentos, idempotencia, validación de entradas.
Justificación (ADR)	Capacidad de explicar el porqué de tus decisiones y sus trade-offs.

8. Opcionales / bonus (suman, no obligatorios)

- Tests (unitarios e/o de integración).
 - CI/CD.
 - Observabilidad (logs estructurados, trazas, dashboards, alarmas).
 - Estrategias avanzadas de resiliencia (idempotencia robusta, *backoff*, colas de descarte).
-

9. Cuenta de AWS para el ejercicio

Para la realización del ejercicio se te **asignará una cuenta de AWS** en una **región específica**, en la cual deberás desplegar y ejecutar toda la solución.

- Trabaja **únicamente** en la cuenta y la región que te sean asignadas.
- Despliega toda la infraestructura con `cdk deploy` en esa región.
- Las credenciales de acceso y los datos de la cuenta y región te serán entregados al inicio del ejercicio.

¡Éxitos! Nos interesa tanto **lo que construyes** como **cómo justificas tus decisiones**.